

```
import pandas as pd
```

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
df=pd.read_csv('/content/drive/MyDrive/Colab Notebooks/Data Science/Dataset/salary.csv')#, encoding='utf-8'
```

```
df.head() # print first 5 rows
```

Unnamed: 0	Timestamp	How old are you?	What industry do you work in?	Job title	If your job title needs additional context, please clarify here:	What is your annual salary? (You'll indicate the currency in a later question. If you are part-time or hourly, please enter an annualized equivalent -- what you would earn if you worked the job 40 hours a week, 52 weeks a year.)	How much additional monetary compensation do you get, if any (for example, bonuses or overtime in an average year)? Please only include monetary compensation here, not the value of benefits.	Please indicate the currency	If "Other," please indicate the currency here:	If your income needs additional context, please provide it here:	
0	0	2021-04-27 11:02:09.743	25-34	Education (Higher Education)	Research and Instruction Librarian	NaN	55000	0.0	USD	NaN	NaN
1	1	2021-04-27 11:02:21.562	25-34	Computing or Tech	Change & Internal Communications Manager	NaN	54600	4000.0	GBP	NaN	NaN
2	2	2021-04-27 11:02:38.125	25-34	Accounting, Banking & Finance	Marketing Specialist	NaN	34000	NaN	USD	NaN	NaN
3	3	2021-04-27 11:02:40.643	25-34	Nonprofits	Program Manager	NaN	62000	3000.0	USD	NaN	NaN
4	4	2021-04-27 11:02:41.793	25-34	Accounting, Banking & Finance	Accounting Manager	NaN	60000	7000.0	USD	NaN	NaN

```
print(df.columns)
```

```
Index(['Unnamed: 0', 'Timestamp', 'How old are you?',
      'What industry do you work in?', 'Job title',
      'If your job title needs additional context, please clarify here:',
      'What is your annual salary? (You'll indicate the currency in a later question. If you are part-time or hourly, please en',
      'How much additional monetary compensation do you get, if any (for example, bonuses or overtime in an average year)? Plea',
      'Please indicate the currency',
      'If "Other," please indicate the currency here: ',
      'If your income needs additional context, please provide it here:',
      'What country do you work in?',
      'If you're in the U.S., what state do you work in?',
      'What city do you work in?',
      'How many years of professional work experience do you have overall?',
      'How many years of professional work experience do you have in your field?',
      'What is your highest level of education completed?',
      'What is your gender?', 'What is your race? (Choose all that apply.)'],
      dtype='object')
```

```
df = df.rename(columns={
    'Unnamed: 0': 'USERID',
    'Timestamp': 'Timestamp',
    'How old are you?': 'Age',
    'What industry do you work in?': 'Industry',
    'Job title': 'Job title',
    'If your job title needs additional context, please clarify here:': 'Job title context',
    'What is your annual salary? (You\'ll indicate the currency in a later question. If you are part-time or hourly, please ent
    'How much additional monetary compensation do you get, if any (for example, bonuses or overtime in an average year)? Please
    'Please indicate the currency': 'Currency',
    'If "Other," please indicate the currency here: ': 'Other currency',
    'If your income needs additional context, please provide it here:': 'Salary context',
    'What country do you work in?': 'Country',
    'If you\'re in the U.S., what state do you work in?': 'State',
    'What city do you work in?': 'City',
    'How many years of professional work experience do you have overall?': 'Overall years of experience',
    'How many years of professional work experience do you have in your field?': 'Years of experience in current job',
    'What is your highest level of education completed?': 'Education level',
    'What is your gender?': 'Gender',
    'What is your race? (Choose all that apply.):': 'Race'
})
```

```
df.head() # print first 5 rows
```

	USERID	Timestamp	Age	Industry	Job title	Job title context	Salary	Compensation	Currency	Other currency	Salary context	Country	
0	0	2021-04-27 11:02:09.743	25-34	Education (Higher Education)	Research and Instruction Librarian	NaN	55000	0.0	USD	NaN	NaN	United States	Mass
1	1	2021-04-27 11:02:21.562	25-34	Computing or Tech	Change & Internal Communications Manager	NaN	54600	4000.0	GBP	NaN	NaN	United Kingdom	
2	2	2021-04-27 11:02:38.125	25-34	Accounting, Banking & Finance	Marketing Specialist	NaN	34000	NaN	USD	NaN	NaN	US	To
3	3	2021-04-27 11:02:40.643	25-34	Nonprofits	Program Manager	NaN	62000	3000.0	USD	NaN	NaN	USA	V
4	4	2021-04-27 11:02:41.793	25-34	Accounting, Banking & Finance	Accounting Manager	NaN	60000	7000.0	USD	NaN	NaN	US	South

```
print(df['Salary'].dtype)
```

```
int64
```

```
# Convert "Salary" string to integers
df['Salary'] = df['Salary'].replace(',', '', regex=True)
# This replaces the comma ',' in 55,000 to 55000

df['Salary'] = df['Salary'].astype(int)
# This converts the column datatypes from object to numbers, in this case integer

df.head()
# This displays the first 5 rows to verify the conversion
```

	USERID	Timestamp	Age	Industry	Job title	Job title context	Salary	Compensation	Currency	Other currency	Salary context	Country	
0	0	2021-04-27 11:02:09.743	25-34	Education (Higher Education)	Research and Instruction Librarian	NaN	55000	0.0	USD	NaN	NaN	United States	Mass
1	1	2021-04-27 11:02:21.562	25-34	Computing or Tech	Change & Internal Communications Manager	NaN	54600	4000.0	GBP	NaN	NaN	United Kingdom	
2	2	2021-04-27 11:02:38.125	25-34	Accounting, Banking & Finance	Marketing Specialist	NaN	34000	NaN	USD	NaN	NaN	US	T
3	3	2021-04-27 11:02:40.643	25-34	Nonprofits	Program Manager	NaN	62000	3000.0	USD	NaN	NaN	USA	V
4	4	2021-04-27 11:02:41.793	25-34	Accounting, Banking & Finance	Accounting Manager	NaN	60000	7000.0	USD	NaN	NaN	US	South

Use df.info to identify which columns has a lot of missing values
df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 28180 entries, 0 to 28179
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   USERID                               28180 non-null  int64
1   Timestamp                             28180 non-null  object
2   Age                                   28180 non-null  object
3   Industry                              28099 non-null  object
4   Job title                             28178 non-null  object
5   Job title context                     7284 non-null   object
6   Salary                                28180 non-null  int64
7   Compensation                           20824 non-null  float64
8   Currency                              28180 non-null  object
9   Other currency                        217 non-null    object
10  Salary context                         3050 non-null   object
11  Country                               28180 non-null  object
12  State                                 23124 non-null  object
13  City                                  28098 non-null  object
14  Overall years of experience            28180 non-null  object
15  Years of experience in current job     28180 non-null  object
16  Education level                       27944 non-null  object
17  Gender                                27998 non-null  object
18  Race                                  27990 non-null  object
dtypes: float64(1), int64(2), object(16)
memory usage: 4.1+ MB
```

```
missing_compensation = df['Compensation'].isna().sum()
print(f"Number of rows with missing values in 'Compensation' column: {missing_compensation}")
```

Number of rows with missing values in 'Compensation' column: 7356

```
def missing_values_summary(df):
    """
    Returns each column name with the number of missing values

    Parameters:
    df (pd.DataFrame): Input dataframe

    Returns:
    pd.Series: Column names with missing value counts
    """
    missing_counts = df.isna().sum()
    return missing_counts[missing_counts > 0] # Return only columns with missing values

# Usage
missing_values = missing_values_summary(df)
print("Columns with missing values:")
print(missing_values)
```

```
Columns with missing values:
Industry            81
Job title           2
Job title context   20896
Compensation        7356
Other currency      27963
Salary context      25130
State              5056
City               82
Education level     236
Gender             182
Race              190
dtype: int64
```

```
# Drop columns 'Job title context', 'Other currency', 'Salary context'
df = df.drop(['Job title context', 'Other currency', 'Salary context'], axis=1)

# Check using df.head()
df.head()
```

	USERID	Timestamp	Age	Industry	Job title	Salary	Compensation	Currency	Country	State	City	Over year experi
0	0	2021-04-27 11:02:09.743	25- 34	Education (Higher Education)	Research and Instruction Librarian	55000	0.0	USD	United States	Massachusetts	Boston	5-7
1	1	2021-04-27 11:02:21.562	25- 34	Computing or Tech	Change & Internal Communications Manager	54600	4000.0	GBP	United Kingdom	NaN	Cambridge	8 - 10
2	2	2021-04-27 11:02:38.125	25- 34	Accounting, Banking & Finance	Marketing Specialist	34000	NaN	USD	US	Tennessee	Chattanooga	2 - 4
3	3	2021-04-27 11:02:40.643	25- 34	Nonprofits	Program Manager	62000	3000.0	USD	USA	Wisconsin	Milwaukee	8 - 10
4	4	2021-04-27 11:02:41.793	25- 34	Accounting, Banking & Finance	Accounting Manager	60000	7000.0	USD	US	South Carolina	Greenville	8 - 10

1.2 Dropping missing values We can drop missing values here using dropna() function. If we do dropna() without specifying any parameter, it will drop rows that has even 1 missing values If we do dropna(thresh=2) it will drop rows that has more than 2 missing values If we do dropna(how=all) it will drop only rows where all values are missing

```
# Number of rows before
rows_before = len(df)
print(f"Rows before: {rows_before}")

# Drop rows with any missing values
df = df.dropna()

# Number of rows after
rows_after = len(df)
print(f"Rows after: {rows_after}")
print(f"Rows dropped: {rows_before - rows_after}")

# Check the result
df.head()
```

Rows before: 28180
 Rows after: 17041
 Rows dropped: 11139

	USERID	Timestamp	Age	Industry	Job title	Salary	Compensation	Currency	Country	State	City	Overall years of experience
0	0	2021-04-27 11:02:09.743	25-34	Education (Higher Education)	Research and Instruction Librarian	55000	0.0	USD	United States	Massachusetts	Boston	5-7 years
3	3	2021-04-27 11:02:40.643	25-34	Nonprofits	Program Manager	62000	3000.0	USD	USA	Wisconsin	Milwaukee	8 - 10 years
4	4	2021-04-27 11:02:41.793	25-34	Accounting, Banking & Finance	Accounting Manager	60000	7000.0	USD	US	South Carolina	Greenville	8 - 10 years
6	6	2021-04-27 11:02:50.507	25-34	Publishing	Publishing Assistant	33000	2000.0	USD	USA	South Carolina	Columbia	2 - 4 years
8	8	2021-04-27 11:03:01.045	45-54	Computing or Tech	Systems Analyst	112000	10000.0	USD	US	Missouri	St. Louis	21 - 30 years

```
# Number of rows before
rows_before = len(df)
print(f"Rows before: {rows_before}")

# Drop rows with more than 2 missing values
df = df.dropna(thresh=2)

# Number of rows after
rows_after = len(df)
print(f"Rows after: {rows_after}")
print(f"Rows dropped: {rows_before - rows_after}")

# Check the result
df.head()
```

Rows before: 17041
 Rows after: 17041
 Rows dropped: 0

	USERID	Timestamp	Age	Industry	Job title	Salary	Compensation	Currency	Country	State	City	Overall years of experience
0	0	2021-04-27 11:02:09.743	25-34	Education (Higher Education)	Research and Instruction Librarian	55000	0.0	USD	United States	Massachusetts	Boston	5-7 years
3	3	2021-04-27 11:02:40.643	25-34	Nonprofits	Program Manager	62000	3000.0	USD	USA	Wisconsin	Milwaukee	8 - 10 years
4	4	2021-04-27 11:02:41.793	25-34	Accounting, Banking & Finance	Accounting Manager	60000	7000.0	USD	US	South Carolina	Greenville	8 - 10 years
6	6	2021-04-27 11:02:50.507	25-34	Publishing	Publishing Assistant	33000	2000.0	USD	USA	South Carolina	Columbia	2 - 4 years
8	8	2021-04-27 11:03:01.045	45-54	Computing or Tech	Systems Analyst	112000	10000.0	USD	US	Missouri	St. Louis	21 - 30 years

```
# Check for each columns. Example is 'Gender'
# Write a code that prints out each columns and their number of unique items
print('--Columns and their number of unique items--')
for column in df.columns:
    n_unique = df[column].nunique()
    print(f'{column} : {n_unique}')

--Columns and their number of unique items--
USERID : 17041
Timestamp : 17041
Age : 7
Industry : 842
Job title : 9429
```

```
Salary : 2396
Compensation : 751
Currency : 7
Country : 116
State : 121
City : 3117
Overall years of experience : 8
Years of experience in current job : 8
Education level : 6
Gender : 5
Race : 41
```

```
# Check the unique items for Gender
print('\n--Unique items in Gender column--')
print(df['Gender'].unique())

--Unique items in Gender column--
['Woman' 'Non-binary' 'Man' nan 'Other or prefer not to answer'
 'Prefer not to answer']
```

```
# Remove redundancy by using df.replace()
# Standardize the gender values based on your actual data
df['Gender'] = df['Gender'].replace({
    'Woman': 'Female',
    'Man': 'Male',
    'Non-binary': 'Other',
    'Other or prefer not to answer': 'Other',
    'Prefer not to answer': 'Other'
})
```

```
# Check whether the values have been replaced or not
print('\n--Gender values after replacement--')
print(df['Gender'].unique())
```